CS683 Programming Assignment II
Task 2: Instruction-Conscious L2 Replacement

Experimental setup

All runs use the ChampSim build from the PA2 repository with 25M warmup and 25M simulated instructions on the three provided traces (Trace1, Trace2, Trace3). The L2 cache has 1024 sets and 8 ways. All prefetchers are off and every other cache uses LRU, so only the L2 replacement policy changes between runs.

- **Baseline:** LRU at the L2.
- **Speedup** = IPC with the policy / IPC with LRU. The "Average" speedup is the geometric mean over the three traces.
- **L2C load MPKI:** MPKI of the `L2C LOAD` line.
- **L2 instruction / data load MPKI:** the `L2C INSTRUCTION LOAD MPKI` and `L2C DATA LOAD MPKI` lines printed by ChampSim (load misses on instruction lines and on data lines per 1000 instructions).

The raw outputs are in `results/task2a` and `results/task2b`. The tables and plots were produced by `scripts/parse_results.py`, `scripts/plot_results.py` and `scripts/plot_task2b.py`.

1 Task 2A: I-RRIP

1.1 Interface change

`update_replacement_state()` now takes an extra argument, `uint8_t isInstruction` (declared in `inc/cache.h`, passed at every call site in `src/cache.cc`). It is computed by `is_instruction_line()`: a request is an instruction line when it is a `LOAD` or `PREFETCH` and `packet.instruction` is set. Translation requests and writebacks are never instruction lines. This is the same rule `fill_cache()` uses to set `block.instruction`, so the policy sees the same classification on a fill, on a hit and at eviction time (`current_set[i].instruction`).

1.2 Design

I-RRIP (`replacement/irrip.l2c_repl`) is SRRIP with 2-bit RRPVs (0 = re-used soon, 3 = evict first), changed in all three parts of the policy:

Part	SRRIP	I-RRIP
Insertion, instruction line	RRPV 2	RRPV 1
Insertion, data line	RRPV 2	RRPV 2
Promotion (hit), instruction line	RRPV 0	RRPV 0
Promotion (hit), data line	RRPV 0	RRPV 1
Promotion on a writeback hit	RRPV 0	no change
Eviction	first line at RRPV 3	first *data* line at RRPV 3, then any line at RRPV 3

Table 1: SRRIP vs. I-RRIP rules.

A hit never raises a line's RRPV (the new value is $\min(\text{old}, \text{target})$).

1.3 Results

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace1	LRU	0.2323	1.0000	17.696	10.338	7.358
Trace1	SRRIP	0.2322	0.9995	17.639	10.562	7.077
Trace1	I-RRIP	0.2322	0.9994	17.649	9.765	7.884
Trace2	LRU	0.2512	1.0000	27.357	18.853	8.504
Trace2	SRRIP	0.2518	1.0026	25.746	17.781	7.965
Trace2	I-RRIP	0.2572	1.0241	21.329	12.204	9.125
Trace3	LRU	0.3821	1.0000	3.954	1.124	2.830
Trace3	SRRIP	0.3817	0.9991	4.039	1.197	2.842
Trace3	I-RRIP	0.3822	1.0003	3.823	0.957	2.866
Average	LRU	0.2885	1.0000	16.336	10.105	6.231
Average	SRRIP	0.2886	1.0004	15.808	9.847	5.961
Average	I-RRIP	0.2905	1.0079	14.267	7.642	6.625

Table 2: Task 2A results. Average: geometric mean for speedup, arithmetic mean otherwise.

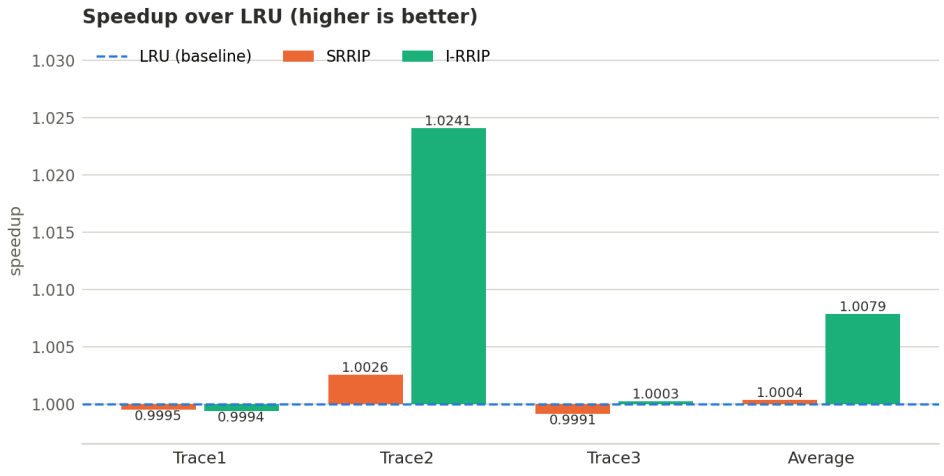


Figure 1: Speedup over LRU.

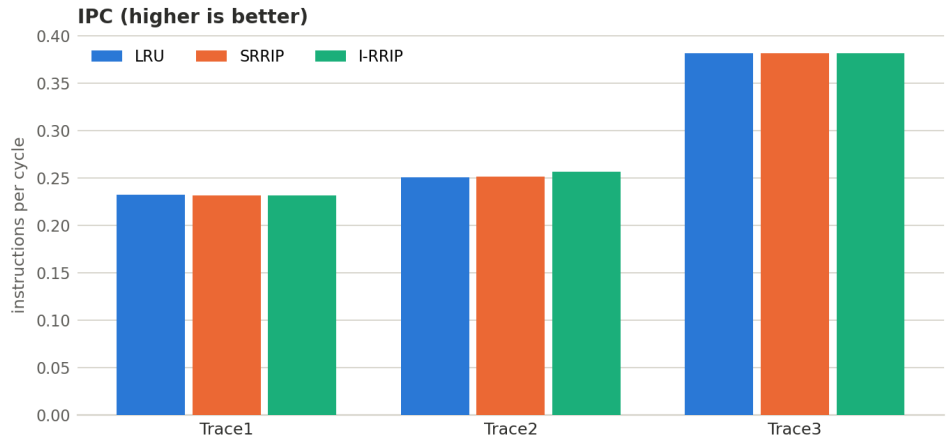


Figure 2: IPC.

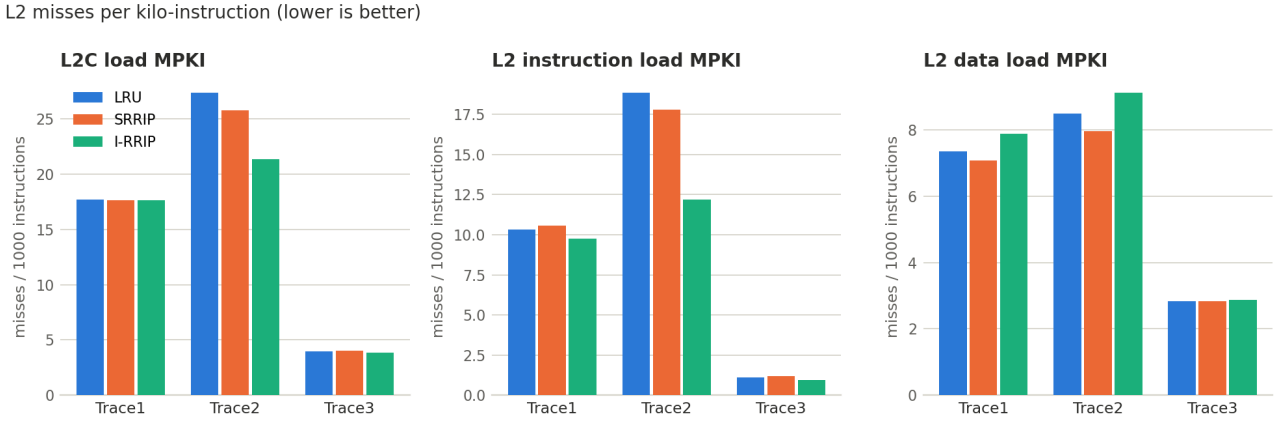


Figure 3: L2C load, L2 instruction and L2 data load MPKI.

Observations:

- I-RRIP cuts the L2 instruction MPKI on every trace: by 24% on average compared with LRU (10.105 → 7.642) and 22% compared with SRRIP. On Trace2, which has the largest code footprint, the drop is 35% (18.853 → 12.204).
- The price is a small rise in data MPKI (6.231 → 6.625 on average). Because the instruction misses fall much more, the total L2C load MPKI still drops by 13% (16.336 → 14.267).
- The gain matters most on Trace2 (2.4% speedup), where the frontend is starved by L2 instruction misses. Trace1 and Trace3 change by less than 0.1%, with SRRIP and I-RRIP both within noise of LRU. On average I-RRIP is 0.79% faster than LRU, against 0.04% for SRRIP.

1.4 Answer: rationale for the insertion, promotion and eviction changes

Why prioritise instruction lines at all. An L2 instruction miss stalls the frontend: no instructions are fetched, so the whole pipeline drains and out-of-order execution cannot hide the latency. A data miss is often overlapped with other independent work. So an instruction hit is worth more than a data

hit. Server-style code footprints do not fit in the L1I, and so the L2 is the last cheap level for code. The goal is therefore to keep code in the L2 even at the cost of a few more data misses.

Insertion: instruction lines at RRPV 1, data lines at RRPV 2. A new code line starts one step safer than a new data line, so it survives one more aging round before it can be evicted. Code has high reuse (loops, functions called repeatedly), so this guess is usually right. We did *not* insert code at RRPV 0 (variant `irrip_aggressive`). That makes a code line used only once as hard to evict as a re-used one, which fills the set with dead code. It raises data MPKI to 6.749 and lowers the speedup to 0.72% (Table 3). Data lines keep SRRIP's RRPV 2, so scan-resistance for streaming data is unchanged.

Promotion: code to 0, data only to 1; writebacks do not promote. On a hit, code goes to the safest value (0). Data goes only to 1, so a re-used data line is still ranked below a re-used code line, and the priority is kept even after hits. Without this, a hot data line would reach 0 and become as safe as code. Two further rules fix problems with plain SRRIP: (i) a `WRITEBACK` hit is the L1D pushing a dirty line down, not a re-use by the program, so it does not promote; (ii) a hit never raises an RRPV, so a hit can never make a line less safe. Variant `irrip_none` applies only these two rules, and they alone raise the speedup from 0.04% (SRRIP) to 0.39%. Writeback hits were giving dirty data lines priority they had not earned.

Eviction: data lines at RRPV 3 first. When the victim search finds several lines at RRPV 3, SRRIP takes the first one. I-RRIP takes the first *data* line among them and only evicts code if every line at RRPV 3 is code. This is a tie-breaker only: it never evicts a data line that is still "younger" than a code line, so useful data is not thrown out early. It is the cheapest change and the most effective one on its own (`irrip_evict_only`, 0.80%).

Why all three changes together. Every change was also tested on its own (the settings are read from environment variables, so one binary runs every variant):

Variant	Changes applied	Speedup (%)	Instr. MPKI	Data MPKI	L2C load MPKI
`rrip` (SRRIP)	none	0.04	9.847	5.961	15.808
`irrip_none`	WB/no-raise rules only	0.39	8.985	5.974	14.959
`irrip_promo_only`	+ data hit to 1	0.51	8.555	6.177	14.732
`irrip_insert_only`	+ code inserted at 1	0.54	8.572	6.239	14.811
`irrip_evict_only`	+ data evicted first	0.80	7.883	6.246	14.129
`irrip_aggressive`	all, code inserted at 0	0.72	7.737	6.749	14.486
`irrip` (final)	all three	0.79	**7.642**	6.625	14.267

Table 3: I-RRIP variants (speedup over LRU, geometric mean; MPKI averaged over the traces).

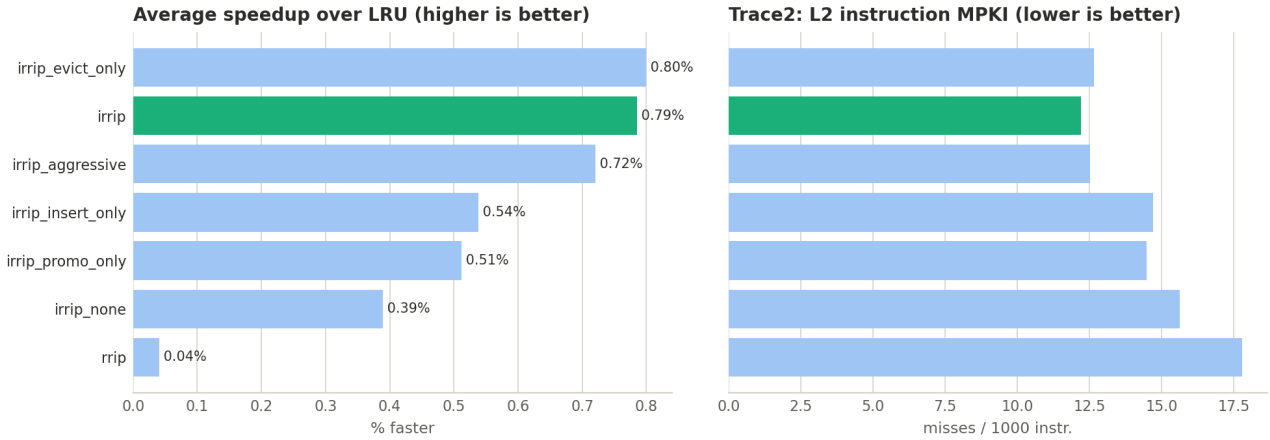


Figure 4: Average speedup of every variant and the L2 instruction MPKI on Trace2.

The final I-RRIP has the **lowest average L2 instruction MPKI** of all variants (7.642), and the lowest on Trace2, where code misses matter most (12.204). Reducing instruction MPKI is the stated goal of the task. `irrip_aggressive` is slightly lower on Trace1 and Trace3 (by under 0.02 MPKI) but loses 0.32 MPKI on Trace2 and costs much more on data. `irrip_evict_only` is 0.01% faster on average, which is within noise. But its instruction MPKI is 3% higher (7.883 vs. 7.642), because without the insertion and promotion changes, code that is not yet at RRPV 3 gets no protection. The three changes work together: insertion protects new code, promotion keeps re-used code ahead of re-used data, and eviction breaks the remaining ties in favour of code. The extra data misses this causes are modest (data MPKI +6% over LRU), and the total load MPKI is close to the best variant. The hardware cost is zero extra storage: the `instruction` bit already exists in each block, and only the comparator logic in the victim search changes.

2 Task 2B: Dynamic I-RRIP

2.1 Design

Dynamic I-RRIP (`replacement/dyn_irrip.l2c_repl`) uses set dueling:

- **Leader sets:** 32 of the 1024 sets (about 3%). They always run plain SRRIP. Leader s satisfies $s \bmod 32 = \lfloor s/32 \rfloor \bmod 32$, so the leaders lie on a diagonal: one per block of 32 sets, at a different offset in each block.
- **Counters (leaders only):** instruction accesses, instruction misses, total accesses and total misses. Writebacks are not counted. A hit is one access, and a fill is one access plus one miss.
- **Decision:** every 1024 leader accesses, compute

$$r = \frac{\text{instr. accesses} + \text{instr. misses}}{\text{total accesses} + \text{total misses}}$$

  and set the follower sets to I-RRIP if $r > T$ (default $T = 0.25$), else SRRIP.
- **Aging:** after each decision the four counters are halved rather than cleared. Old behaviour then fades out gradually, and one unusual window cannot flip the mode by itself.
- **Follower sets** use the Task 2A I-RRIP rules or the SRRIP rules depending on the current mode. The simulation starts in I-RRIP mode.

Storage: one mode bit, four counters and a 10-bit window counter. Whether a set is a leader is computed from the set index, so the leaders need no storage.

2.2 Results

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace1	LRU	0.2323	1.0000	17.696	10.338	7.358
Trace1	SRRIP	0.2322	0.9995	17.639	10.562	7.077
Trace1	I-RRIP	0.2322	0.9994	17.649	9.765	7.884
Trace1	Dyn-I-RRIP	0.2321	0.9991	17.651	9.788	7.864
Trace2	LRU	0.2512	1.0000	27.357	18.853	8.504
Trace2	SRRIP	0.2518	1.0026	25.746	17.781	7.965
Trace2	I-RRIP	0.2572	1.0241	21.329	12.204	9.125
Trace2	Dyn-I-RRIP	0.2568	1.0223	21.724	12.631	9.093
Trace3	LRU	0.3821	1.0000	3.954	1.124	2.830
Trace3	SRRIP	0.3817	0.9991	4.039	1.197	2.842
Trace3	I-RRIP	0.3822	1.0003	3.823	0.957	2.866
Trace3	Dyn-I-RRIP	0.3818	0.9993	3.931	1.064	2.868
Average	LRU	0.2885	1.0000	16.336	10.105	6.231
Average	SRRIP	0.2886	1.0004	15.808	9.847	5.961
Average	I-RRIP	0.2905	1.0079	14.267	7.642	6.625
Average	Dyn-I-RRIP	0.2902	1.0068	14.436	7.827	6.608

Table 4: Task 2B results (Dyn-I-RRIP with threshold 0.25, 32 leader sets).

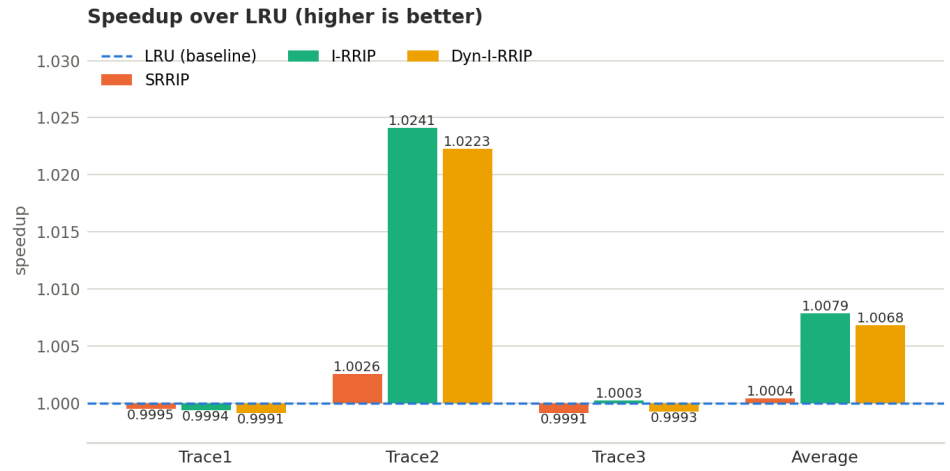


Figure 5: Speedup over LRU.

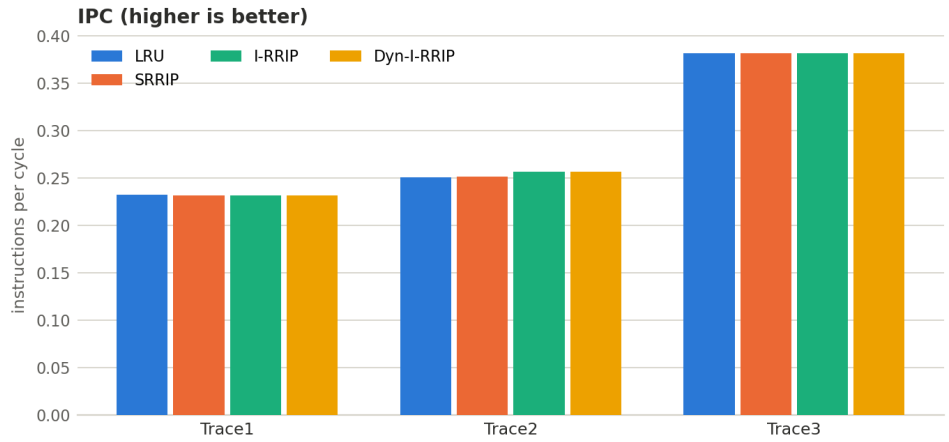


Figure 6: IPC.

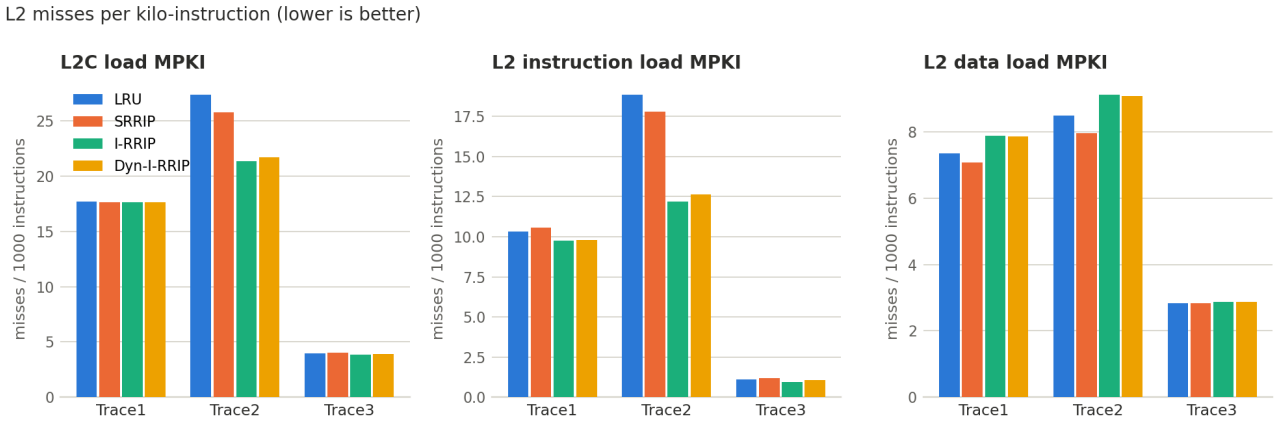


Figure 7: L2C load, L2 instruction and L2 data load MPKI.

Dyn-I-RRIP keeps most of the I-RRIP benefit: 0.68% average speedup and a 23% lower instruction MPKI than LRU. It does not beat static I-RRIP on these three traces, for two reasons:

- On Trace1 and Trace2 the measured ratio is always well above the threshold (average $r = 0.44$ and 0.60), so the followers stay in I-RRIP mode (100% and 94% of accesses). The 3% of sets that are leaders run SRRIP and lose the I-RRIP benefit. This is the fixed cost of set dueling.
- On Trace3 the ratio is close to the threshold (average $r = 0.31$), and the policy uses SRRIP for 42% of accesses. But SRRIP is not better than I-RRIP on this trace either (0.9991 vs. 1.0003), so switching does not help.

None of the three traces has a long data-dominated phase where I-RRIP hurts. The dynamic policy is a safety net for such workloads, and here it costs 0.1% when the net is not needed.

2.3 Sensitivity study

The threshold T was swept over five values with 32 leader sets (`dyn_t*` runs), and the number of leader sets over 16, 32 and 64 with $T = 0.25$ (`dyn_l*` runs).

T	Speedup (geo)	Instr. MPKI	Data MPKI	L2C load MPKI	% accesses in I-RRIP mode Trace1	Trace2	Trace3
0.10	1.0077	7.701	6.610	14.311	100.0	100.0	100.0
0.20	1.0069	7.823	6.607	14.430	100.0	95.8	71.9
0.25	1.0068	7.827	6.608	14.436	100.0	93.7	57.5
0.30	1.0068	7.856	6.614	14.470	100.0	93.7	42.3
0.40	1.0065	7.927	6.571	14.498	88.4	91.5	42.5

Table 5: Threshold sensitivity (MPKI averaged over the traces).

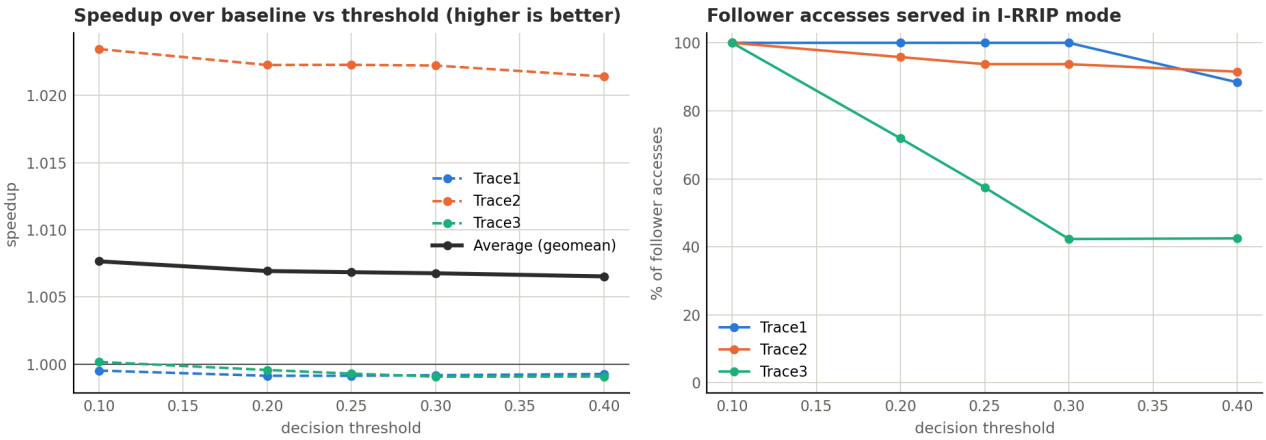


Figure 8: Speedup and share of follower accesses in I-RRIP mode vs. threshold.

Leader sets	Speedup (geo)	Instr. MPKI	Data MPKI	L2C load MPKI
16	1.0069	7.840	6.601	14.441
32	1.0068	7.827	6.608	14.436
64	1.0068	7.874	6.599	14.473

Table 6: Leader-set sensitivity ($T = 0.25$).

A lower threshold keeps the followers in I-RRIP mode more often. The speedup moves monotonically, from 1.0077 at $T = 0.10$ (always I-RRIP, no switches) down to 1.0065 at $T = 0.40$, while data MPKI improves only slightly at $T = 0.40$. The whole range spans just 0.12%, so the design is not fragile with respect to T. The number of leader sets has no measurable effect between 16 and 64.

2.4 Answer: rationale for the number of leader sets and the decision logic

Number of leader sets: 32. Leader sets are a trade-off between two costs.

- *Too few leaders* give a small, noisy sample. A few hot sets could then decide for the whole cache. With 1024 leader accesses per decision, fewer leaders also means a window takes longer to fill, so the policy reacts more slowly to a phase change.
- *Too many leaders* waste performance. Leaders always run SRRIP, so every extra leader is a set that never gets the I-RRIP benefit when I-RRIP is the right choice. This is the cost visible on Trace1 and Trace2.

32 leaders is about 3% of the 1024 sets. That is the value used in the original set-dueling work (DIP, Qureshi et al., ISCA 2007), which showed that 32 sets are enough to estimate the behaviour of the whole cache. Our sweep agrees: 16, 32 and 64 leaders give the same speedup to within 0.01%. We chose 32 rather than 16 because it gives twice the sample size (256 lines) for a negligible cost, which matters more for workloads with uneven set usage than for these three traces. The leaders follow a diagonal pattern instead of every 32nd set, so they do not all share the same low-order index bits. Workloads whose strides map to particular sets would otherwise bias the sample.

Decision logic.

- *The ratio.* We use the formula from the hint, $(I_{acc} + I_{miss})/(T_{acc} + T_{miss})$. It measures how much of the L2 demand comes from code. Counting misses as well as accesses gives extra weight to the type that is actually missing. A workload with many code accesses that all hit is not code-critical, but one with many code misses is. Writebacks are excluded because they are not program demand and would inflate the data side.

- *Leaders run SRRIP.* The ratio is measured under the neutral policy. If leaders ran I-RRIP, the protected code would miss less, the ratio would drop, and the policy would switch away from I-RRIP exactly when it is working. Measuring under SRRIP avoids this feedback loop.

- *Window of 1024 leader accesses with halving.* 1024 accesses give a stable ratio (Trace1 makes 26 decisions in the measured region, Trace2 46, Trace3 6). Halving the counters, rather than clearing them, gives an exponentially weighted history. A short burst cannot flip the mode, but a real phase change takes over within a few windows. The low number of mode switches (0 to 4 per run) shows the policy does not oscillate.

- *Threshold 0.25.* The sweep shows that a lower T is slightly faster on these traces. But $T = 0.10$ never leaves I-RRIP mode on any trace, so it simply turns Dyn-I-RRIP into static I-RRIP and removes the protection the dynamic policy exists for. $T = 0.25$ costs only 0.09% against $T = 0.10$. It still switches to SRRIP when code makes up less than a quarter of L2 demand, the case where giving priority to code would hurt data most. We therefore keep $T = 0.25$ as the default.

Limitation and possible improvement. The ratio measures how much of the L2 demand is code. It does not directly measure which policy misses less. Trace3 shows the gap: the policy switches to SRRIP for 42% of accesses even though SRRIP is not better there. A natural extension is a true duel: one group of leaders runs SRRIP and another runs I-RRIP, and a single saturating counter records which group misses less, as in DIP. This would pick the better policy directly instead of predicting it from the traffic mix.